Mini Spotify

A Spotify-like music streaming application built in pure Java with socket programming, running on a 3-node distributed backend

---

1. Project Title

Mini Spotify – A Spotify-Like Distributed Music Streaming Application in Pure Java (Socket Programming).

SimpleStream is a music streaming application with two front ends, a website and a terminal client. Both talk to a cluster of three socket-server nodes. It uses no frameworks and no external libraries; everything communicates through *java.net.Socket* and *java.net.ServerSocket*.

2. Project Overview & Objectives

The application lets a user sign up with an email, log in, search a catalogue of 18 songs (6 Hindi, 6 English, 6 Telugu), play them, like songs, build playlists and see recently played tracks. Accounts and user data are saved to disk, so they are still there after a restart. The music player itself is deliberately ordinary. The main purpose of the project is what sits underneath it: a small distributed system that makes core distributed-computing ideas visible and demonstrable.

Objectives

- Build a complete, working music application using only Java's standard socket API: no Spring, no servlets, no database server.
- Demonstrate the main distributed-systems paradigms in one project: client-server, load balancing, replication, fault tolerance, logical clocks, concurrency, stateless design and persistence.
- Make the distributed behaviour observable: the website sidebar shows which node served each request, a button crashes a node, and the terminal prints the logical clock.
- Keep the design small enough that a reader can follow one request from the screen to the disk and back.

3. How to Run the Application

Prerequisite: JDK 11 or newer (check with *javac -version*). Open a terminal inside the project folder, the one that contains *run.bat*, *run.sh*, *client/*, *server/* and *public/*. The whole project runs with two commands.

Step	Windows	Linux / Mac	What it does
1. Build	*run.bat build*	*./run.sh build*	Compiles all Java sources into *server/out*. Run it the first time and after any code change. It prints *Build OK*.
2. Start	*run web*	*./run.sh web*	Starts the 3-node backend and the website. Open http://localhost:8080. The first run also generates 18 WAV songs (about 5 seconds).
3. Terminal (optional)	*run terminal*	*./run.sh terminal*	Starts the terminal client. It can run together with the website.
Backend only (optional)	*run backend*	*./run.sh backend*	Starts only the 3 backend nodes, for use with the terminal client alone.

- On Windows the backend opens in a second window named “Spotify Backend”. On Linux and Mac it runs in the background and stops when you press Ctrl+C.
- The terminal client needs the backend, so start *run web* (or *run backend*) first.
- Without the scripts: *java -cp server/out spotify.MusicServer* (backend) and *java -cp server/out spotify.SpotifyWeb* (website).
- “Address already in use” means an old backend is still running. Close it and start again.

Quick start on the website: open http://localhost:8080, choose the **Sign Up** tab, enter email, username and password, press **Create account**, then log in with the **email and password**.

4. Project Files & Technology Stack

File	Role
Client side (*client/src/simplestream/*)	
Balancer.java	Client-side round-robin load balancing and failover over sockets. Used by both the website and the terminal.
SpotifyWeb.java	Mode 1: a small HTTP server built on *ServerSocket*. It serves *public/* (HTML, CSS, JS), streams audio and handles */api/** calls.
SpotifyTerminal.java	Mode 2: command-line Spotify that plays audio with *javax.sound*.
Server side (*server/src/simplestream/*)	
MusicServer.java	The backend: 3 socket-server nodes (ports 5001–5003), accounts, saving and loading, the text protocol, replication and the Lamport clock.
Catalog.java, *Song.java*	The 18-song catalogue (6 Hindi, 6 English, 6 Telugu) and the song model.
AudioGenerator.java	Creates a different tune for each song (own scale, tempo and key) as *server/data/audio/<id>.wav*.
Website (*public/*)	
index.html, *css/style.css*, *js/app.js*	The page (login card, sidebar, player bar), the Spotify-style dark theme, and the login, rendering and player logic that calls */api/...*.
Scripts and data	
run.bat, *run.sh*	The *build*, *web*, *terminal* and *backend* commands.
server/data/	*audio/* (song files), *users.db* (accounts), *userdata.db* (likes, playlists, history, play counts).

Technology stack: Java (JDK 11+) with *java.net.Socket* / *ServerSocket* for all networking; threads and *synchronized* for concurrency; *javax.sound* for terminal playback; SHA-256 (*MessageDigest*) for passwords; plaintext files for storage; and plain HTML, CSS and JavaScript (*fetch()*) for the website. There are no frameworks and no third-party libraries.

5. Accounts – Sign Up, Login & Logout

- **Sign Up** needs an email, a username and a password. The email must be valid and not already used. The username must be 3–15 letters, digits or _ and not already taken. The password needs at least 3 characters.
- After signing up, the website shows an "Account created" message and switches to the **Log In** tab.
- **Log In** on the website uses **email + password**. **Log out** is a button in the sidebar. The browser remembers who is logged in using *localStorage*, so a page refresh does not log the user out.
- Passwords are stored only as **SHA-256 hashes**, never as plain text. Each user has their own Library (liked songs), Playlists and Recently Played.
- The terminal client uses the older style: *signup <user> <pass>* and *login <user> <pass>*. Terminal accounts have no email, so they cannot be used for email login on the website, but they can still log in with their username.

Login flow: the browser sends the email and password to *SpotifyWeb*, which sends *LOGIN<TAB>email<TAB>password* to a node. The node finds the account for that email, checks the password hash and replies with the username (*U|username*). From then on, every request carries the username, and each user's data is kept separately.

6. Data Persistence

Every write (sign up, like, create, add, remove or delete a playlist, play) is saved to disk right after it is applied. When the backend starts, all three nodes load these files, so every node has the same data again.

Data	Saved in	Survives restart?

Accounts (email, username, password hash)	*server/data/users.db*	Yes
Liked songs (Library)	*server/data/userdata.db*	Yes
Playlists and their songs	*server/data/userdata.db*	Yes
Recently Played	*server/data/userdata.db*	Yes
Play counts (“Most Played”)	*server/data/userdata.db*	Yes

To **reset everything**, stop the app and delete *users.db* and *userdata.db*. They are created again automatically.

7. Using Custom Music

Song files are named by the song's **id**, which is its position in *Catalog.java* (the first song is 1, the second is 2, up to 18).

1. Rename your mp3 to the id, for example *7.mp3*, and put it in *server/data/audio/*.
2. **Delete the matching `7.wav`.** The website plays the *.wav* first and only falls back to the *.mp3* when the *.wav* is missing.
3. Change the title and artist for that song in *Catalog.java*, run *run build* and restart.

To add **new** songs, add lines at the **end** of the *SONGS* array in *Catalog.java*; they become ids 19, 20 and so on, with files *19.mp3*, *20.mp3*. Never insert a song in the middle of the list, because that changes the ids of later songs and mixes up saved likes and playlists. The website plays mp3 or wav; the terminal client plays only WAV.

8. Website Features

- A Spotify-style login card with **Log In / Sign Up** tabs and a **Log out** button.
- Song cards with cover art and a hover ▶ button on **Home**, **Search**, **Library** (♥), **All Songs** and **Playlists**. Home leads with a “Trending Songs 2026” section.
- Playlists: create, open and delete; “+ Playlist” on any song adds it, and ✕ removes it.
- A player bar with Play/Pause, Next, Previous and a seek bar, visible on every screen.
- A sidebar that shows the 3 backend nodes (green when up, red when down, plus which node served the last request) and a “Crash last node” button for the fault-tolerance demo.

9. Terminal Client Features & Commands

The terminal client offers the same functions as the website from a command line, and adds two commands for watching the cluster (*nodes*, *crash*). Its first screen asks you to sign up (for example *signup alice 1234*) or log in (*login alice 1234*). After each request it prints which node served it and the current logical clock.

Group	Commands	Purpose
Account	*signup <user> <pass>*, *login <user> <pass>*, *logout*, *whoami*	Create an account, log in or out, show the current user
Browse	*home*, *songs*, *search <text>*, *library*, *like <id>*	Home page, all songs, search, liked songs, like a song
Playlists	*playlists*, *open <name>*, *create <name>*, *delete <name>*, *add <songId> <playlist>*, *remove <songId> <playlist>*	Manage playlists and their songs
Playback	*play <id>*, *pause*, *resume*, *next*, *prev*, *now*	Play and control a song (WAV only)
Cluster	*nodes*, *crash <port>*, *help*, *quit*	Show node status, simulate a node failure, help, exit

10. System Architecture

The system has four layers: the clients, a gateway for the website, a client-side balancer, and the backend cluster with its storage. There is no central coordinator; the three backend nodes are equals.

```
 Browser --HTTP--> SpotifyWeb --\
                                >-- Balancer --TCP--> MusicServer node 5001 \
 SpotifyTerminal -------------/    (round robin)     MusicServer node 5002  } replicate
                                                     MusicServer node 5003 /   writes
                                                           |
saved to server/data/users.db + userdata.db
```

Layer	Responsibility
Clients	The browser (HTML/CSS/JS) and *SpotifyTerminal*. They only send requests and show replies; they hold no user data.
Gateway (*SpotifyWeb*)	A hand-written HTTP server on port 8080. It serves the static files, streams audio (*/audio/<id>*) and turns each */api/...* call into one socket command for the cluster.
Balancer	Chooses which node receives each request (round robin) and retries on another node if one does not answer. It is a library class linked into both clients, so there is no separate balancer process to fail.
Backend cluster (*MusicServer*)	Three identical nodes on ports 5001–5003. Each holds all accounts and user data in memory, answers reads, applies writes and replicates them to its peers.
Layer	**Responsibility**
Storage	Two text files, *users.db* and *userdata.db*, written after every write and loaded by all nodes at startup.

Example – liking a song: (1) the user clicks ♥ in the browser; (2) the browser calls */api/like*; (3) *SpotifyWeb* asks the Balancer to send *LIKE<TAB>alice<TAB>7*; (4) the Balancer picks the next node, say 5002; (5) node 5002 increases its clock, applies the like and saves it to disk; (6) it forwards the write to nodes 5001 and 5003 as a *REPL* message; (7) it replies *OK|5002|clock*, and the page shows the filled heart. Any node can now return the same Library.

11. Distributed System Architecture and Communication

11.1 Topology

The backend is a **symmetric (peer-to-peer) cluster**. All three nodes run the same code, hold the same data, and can serve any request. *MusicServer.main* starts the three nodes in one program on ports 5001, 5002 and 5003 of localhost. Each node is an ordinary *ServerSocket* accept loop that starts a new thread for every connection. There is no master node, so there is no leader to lose.

11.2 Communication paths

Path	Transport	Purpose
Browser → *SpotifyWeb*	HTTP over TCP	Page loads, */api/** calls and audio streaming (with HTTP Range support, so seeking works).
Client → node (via Balancer)	TCP socket, text protocol	All application requests: login, search, like, playlists, play, next, previous.
Node → node	TCP socket, *REPL* message	Forwarding each write to the other two nodes.
Balancer / sidebar → node	TCP socket, *PING*	Health check that drives the green and red node dots.

11.3 Message format

The protocol is plain text, with **one short TCP connection per request**. The client opens a socket, sends one line, reads lines until *END*, and closes the socket.

```
Request :  COMMAND<TAB>arg<TAB>arg \n
Reply   :  OK|<node port>|<lamport clock>      <- header
           #Section title                       <- optional
           S|id|title|artist|language|liked|plays   <- data lines
END
Replicate: REPL<TAB><clock><TAB>LIKE<TAB>alice<TAB>7
```

Write commands are *SIGNUP*, *LIKE*, *PLCREATE*, *PLDELETE*, *PLADD*, *PLREMOVE* and *PLAY*. Reads (*SONGS*, *SEARCH*, *HOME*, *LIBRARY*, *PLAYLISTS*, *PLSONGS*, *NEXT*, *PREV*) change nothing, so they are served by one node without any replication.

11.4 Time-outs

Every connection has a time-out so a dead node cannot hang the system. The Balancer waits **800 ms** to connect and **4 s** for a reply, and a node that fails either test counts as down for that request. When a node replicates a write, it

waits **500 ms** to connect and **1 s** for a reply, and a peer that does not answer is skipped, so one dead node cannot block a write.

12. Distributed Paradigms Exhibited

The table lists each paradigm, where it appears in the project, and how to see it working.

Paradigm	Where it is exhibited	How to observe it
1. Client-Server	The website and the terminal are clients. The three *MusicServer* nodes are servers. They talk over sockets using a request/reply protocol.	Any action in the app is one request and one reply.
2. Load balancing (round robin)	*Balancer.send()* picks the next node for every request, using a counter incremented inside a *synchronized* block.	The sidebar's “last served” marker moves 5001 → 5002 → 5003. The terminal prints the serving node.
3. Replication (write-all)	*MusicServer.handle()* applies a write locally, then *replicate()* sends a *REPL* message to the other nodes. Replicated writes: sign up, like, play, and every playlist change.	Like a song, then read the Library again. Whichever node answers shows the same data.
Paradigm	**Where it is exhibited**	**How to observe it**
4. Fault tolerance and failover	*Balancer.send()* catches the connection error and tries the next node. *replicate()* skips a peer that is down.	Click “Crash last node” or type *crash 5001*. The app keeps working and the node's dot turns red.
5. Lamport logical clock	Each node keeps a *clock* counter in *MusicServer*. A write adds 1; a replica sets *clock = max(own, received) + 1*.	The terminal shows “logical clock N” and the backend log prints *clock=* for each write.
6. Multithreading / concurrency	A new thread serves each connection. *synchronized* protects the in-memory data and the file writes.	Several browser tabs and the terminal can be used at the same time without corrupting data.
7. Stateless requests	*NEXT* and *PREV* carry the current song id and playlist. The username travels with every request. No node stores a session.	Next/Previous work even when consecutive requests are answered by different nodes.
8. Persistence (durability)	Accounts and user data are written to *users.db* and *userdata.db* after every write and reloaded when the nodes start.	Stop and restart the backend. Accounts, likes and playlists are still there.

13. Distributed Algorithms and Mechanisms

13.1 Round-robin load balancing with failover

The Balancer keeps a shared counter. For each request it starts at node *counter % n* and, if that node cannot be reached, tries the next one, wrapping around until every node has been tried once.

```
start = counter++ % n                       // inside a synchronized block
for i = 0 .. n-1:
    node = nodes[(start + i) % n]
    try:  return ask(node, command)         // remember node as 'last served'
catch IOException:  log and try the next node throw "All backend nodes are
down"
```

Because the starting point moves by one for every request, load is spread evenly, and because the loop continues past a failed node, a single failure is invisible to the user. An error appears only when all three nodes are down.

13.2 Write-all replication

1. A client sends a write to any node (chosen by the Balancer).
2. Inside a *synchronized* block that node updates its clock, applies the write to its in-memory data and saves the files.
3. **After** releasing the lock, it sends *REPL<TAB>clock<TAB>original command* to each other node (500 ms connect, 1 s read). A peer that is down is skipped.
4. Each peer applies the same command and merges the clock. A message that arrives as *REPL* is **not forwarded again**, which prevents endless replication loops.
5. The first node replies to the client with *OK|port|clock*.

Replication happens outside the lock so that a slow or dead peer never blocks the node's own data. Since every node holds a full copy, reads need no coordination and can be answered by any node. The consistency model is simple: **write-all, best effort**, which is strong while all nodes are healthy and weaker once a node has crashed (see section 15).

13.3 Lamport logical clock

Real clocks on different machines disagree, so the nodes order events with a **logical clock** instead. Each node holds one counter, and the rules are:

- **Local write:** *clock = clock + 1*.
- **Receiving a `REPL` message with clock c:** *clock = max(clock, c) + 1*.
- **Reply header:** the clock after the operation is returned as *OK|port|clock* and shown by the terminal.

For example, if node 5002 (clock 4) handles a like, its clock becomes 5 and it sends *REPL … 5*. A peer whose clock was 2 sets it to *max(2, 5) + 1 = 6*, and a peer whose clock was 7 sets it to *max(7, 5) + 1 = 8*. The result is that an effect always has a larger clock than its cause, so the order of related events is the same on every node.

13.4 Failure detection and failure simulation

Failure is detected by the failure itself: a connect or read error in the Balancer, or in *replicate()*, marks a node as unreachable for that request. The sidebar also sends *PING* to each node to draw the green and red dots. To demonstrate failures, the special *CRASH* command makes a node close its server socket, which behaves like a real node going down.

13.5 Concurrency control and safe persistence

- Each node's data is modified only inside *synchronized (this)*, and the lock is held only while local data changes.
- File writes are guarded by a static lock on *MusicServer.class*, because the three nodes share the same two files.
- Data is first written to a temporary file and then moved over the real file, so a crash in the middle of saving cannot leave a half-written *userdata.db*.

13.6 Stateless request design

A node never remembers a user between requests. Every request includes the username, and *NEXT* / *PREV* include the current song id and playlist, so the node can compute the answer from the request alone. This is what makes it safe for the Balancer to send consecutive requests from one user to different nodes.

14. Implementation Details, Testing & Results

14.1 Implementation details

- **Data model:** *Song* holds id, title, artist, language and a play count. A *User* holds the password hash, email, a set of liked song ids, a map of named playlists (every account starts with “My Favourites”) and a list of recently played ids.
- **Song ids:** a song's id is its position in *Catalog.SONGS*, starting at 1. Audio files and cover images are found by this id.
- **Gateway routes:** *SpotifyWeb* maps API names such as *home*, *search*, *like*, *playlists*, *play*, *next* and *prev* to socket commands, converts the node's reply lines into JSON, and also serves */audio/<id>* with HTTP Range support.
- **Audio:** *AudioGenerator* synthesises 18 different WAV tunes on first run, so the project works without any music files.
- **Front end:** *app.js* renders every screen from the JSON the gateway returns and polls the node status for the sidebar.

14.2 Testing and results

The system is tested by running the complete application and exercising each feature end to end, on the website and in the terminal client. The scenarios below cover the functions and the distributed behaviour.

Scenario	Steps	Expected result
Account creation	Sign Up with email, username, password; then log in with email and password.	Account created; login succeeds; duplicate email or username is rejected.
Search and playback	Search by title, artist or language; play, pause, next, previous, seek.	Matching songs appear; audio plays and the player bar responds.

Likes and playlists	Like songs; create a playlist; add and remove songs; delete the playlist.	Library and playlists update immediately and per user.
Load balancing	Click through several pages and watch the sidebar.	“Last served” moves 5001 → 5002 → 5003.
Replication	Like a song, then repeat reads so different nodes answer.	Every node returns the same Library.
Failover	Press “Crash last node” (or *crash 5001*) and keep using the app.	No error is shown; the node's dot turns red; requests go to the other nodes.
Persistence	Stop and restart the backend, then log in.	Accounts, likes, playlists and history are still present.

Results: all features behave as described, and each of the eight paradigms can be shown live. The most useful demonstrations are the moving “last served” marker (load balancing), the crash button (failover) and the logical clock printed in the terminal.

15. Limitations & Future Enhancements

Limitations (kept simple for the demo):

- A crashed node is **not re-synced** when it is restarted; the whole backend must be restarted so that every node reloads the files. While a node is down it misses writes, so the replicas can differ.
- Data is stored in **simple text files** shared by the three nodes, not in a real database. The three nodes also run in one program on one machine.
- Passwords use **plain SHA-256 without a salt**.
- The **username is sent with each request** instead of a session token. **Future enhancements:**
- Let a restarted node catch up from a healthy peer (state transfer or a replay log).
- Replace the text files with a real database, and run nodes on separate machines.
- Use salted, slow password hashing and session tokens.
- Add recommendations and a larger, user-uploaded catalogue.

16. Conclusion

SimpleStream shows that a familiar application can be used to explain distributed systems. Built with nothing but Java sockets, it combines a website and a terminal client with a three-node cluster that balances load, replicates every write, survives the loss of a node, orders events with a Lamport clock, serves many users at once and keeps its data across restarts. Each idea is small enough to read in one method, and each can be demonstrated live from the interface. The known limitations are deliberate simplifications and point directly to the next steps: node recovery, a real database and stronger consistency.

Sample Login Credentials

The application has **no pre-loaded accounts**; every account is created through Sign Up. Use the following sample values (the terminal example is the one given in the project README):

Where	Credentials	Notes
Website – Sign Up	Email *alice@example.com* Username *alice* Password *1234*	Any valid unused email, a 3–15 character username and a password of 3+ characters.
Website – Log In	Email *alice@example.com* Password *1234*	The website logs in with email and password, not username.
Terminal – create	*signup alice 1234*	Terminal accounts use username and password only (no email).
Terminal – log in	*login alice 1234*	Terminal-created accounts cannot be used for email login on the website.

- **23MCCE22 – K AMULYA**